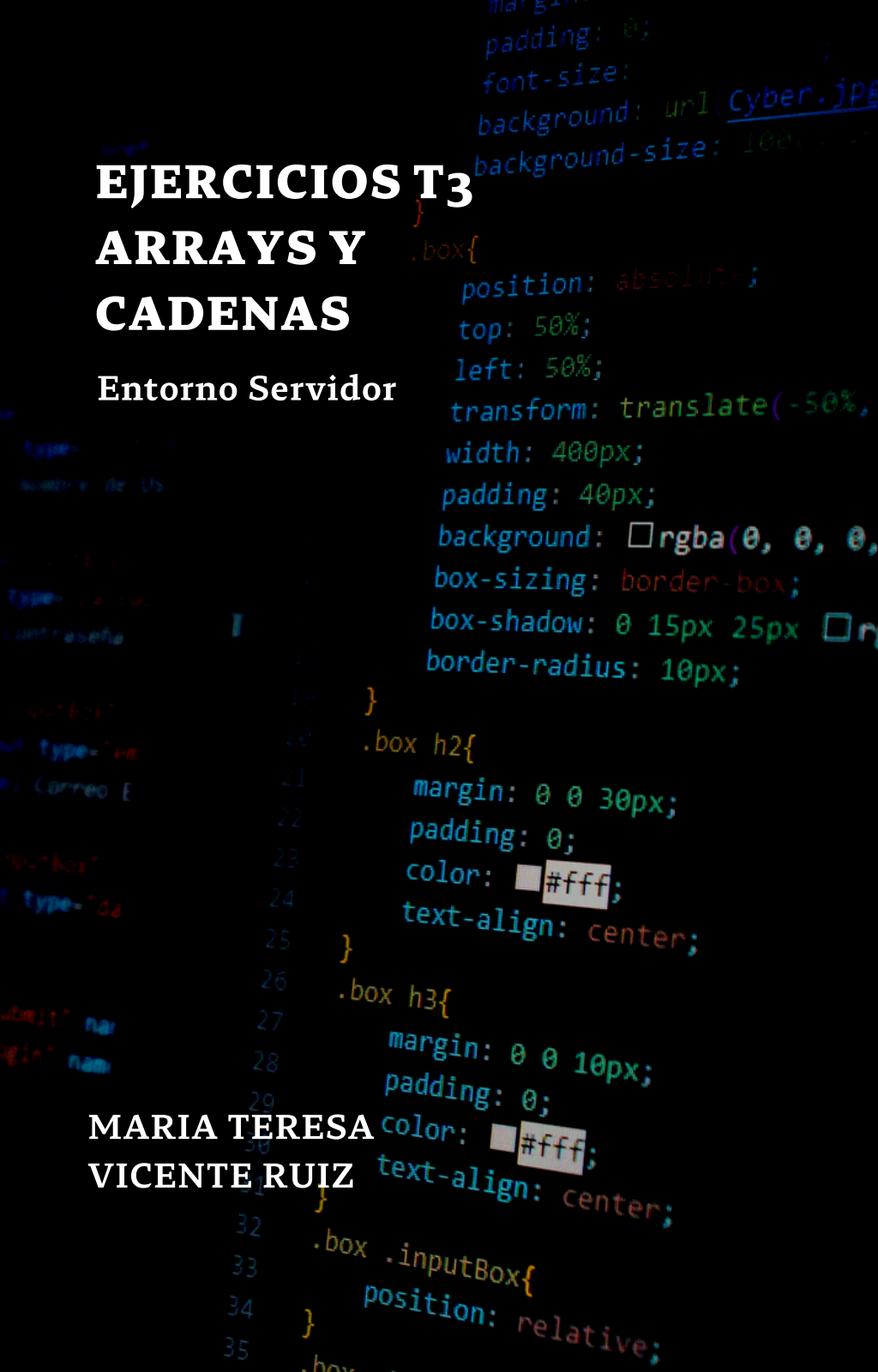
EJERCICIOS T3 ARRAYS Y CADENAS

Entorno Servidor



MARIA TERESA VICENTE RUIZ

ÍNDICE



1. **CIFRAR Y DESCIFRAR** 3
   - **Manual de usuario** 3
   - **Manual técnico** 4
2. **CIFRAR Y DESCIFRAR VERSIÓN 2** 7
   - **Manual de usuario** 7
   - **Manual técnico** 9
3. **RESERVA DE ASIENTOS DE AVIÓN** 10
   - **Manual de usuario** 10
   - **Manual técnico** 12





Maria Teresa Vicente Ruiz

DOCUMENTACIÓN

1. CIFRAR Y DESCIFRAR

Esta aplicación es un cifrado de mensajes. El usuario introduce el mensaje que desee (deben ser todo letras) y la semilla. La semilla debe ser un número positivo. Finalmente se muestra el mensaje cifrado.

1. 1. MANUAL DE USUARIO

La aplicación realizada le pide al usuario el mensaje a introducir y la semilla que desea para proceder a cifrar el mensaje. A su vez también le da la opción de elegir si quiere cifrar o descifrar el mensaje.



Una vez haya introducido el mensaje y la semilla (necesario ambos campos) tendrá que pulsar el botón de cifrar y aparecerá el resultado del mensaje cifrado debajo de los botones. Y tras cifrar el mensaje si vuelve a darle al botón de descifrar le aparecerá el mensaje descifrado de nuevo.





Si en el mensaje se introduce cualquier otro carácter que no sean letras saltará un mensaje de no válido. Al igual que si intentas introducir una letra en la semilla el formulario no te dejará ya que es un input type number.





Se mostrarán los errores en color rojo para que se distingan con mayor facilidad.

El usuario tendrá que introducir valores nuevos válidos en los campos y pulsar nuevamente el botón para cifrar o descifrar el mensaje que desee.

1. 2. MANUAL TÉCNICO

Este ejercicio de cifrado y descifrado lo hemos dividido en 3 archivos: index.php, cifrar.php y descifrar.php. Al principio del index.php tenemos un pequeño código php donde recogemos el mensaje de error o éxito dependiendo de lo que mande el archivo de cifrar o de descifrar. Si el usuario pulsa el botón de cifrar la aplicación se dirige a este archivo para validar el mensaje, y si pulsa al botón de descifrar pues se dirige a este otro.

```php
<?php

$error = $_GET['error'] ?? "";
$exito = $_GET['exito'] ?? "";

?>

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0"
```

Dentro del form, los input type recogen el value mediante $_GET de php para que se mantenga el valor introducido por el usuario por si quiere volver a descifrar ese mismo mensaje.

```php
<body>
    <h1>Cifrar y descifrar mensajes</h1>

    <form method="post">
        <label for="mensaje">Mensaje:</label><br>
        <input type="text" id="mensaje" name="mensaje"
        value="<?= $_GET['mensaje'] ?? '' ?>" required><br>

        <label for="semilla">Semilla:</label><br>
        <input type="number" id="semilla" name="semilla"
        value="<?= $_GET['semilla'] ?? '' ?>" required><br><br>

        <button class="btn-cifrar" type="submit" formaction="./cifrar.php">Cifrar</button>

        <button class="btn-descifrar" type="submit" formaction="./descifrar.php">Descifrar
    </form>
    <br>
        <?php if (!empty($error)): ?>
        <div class="resultado alerta-error">
            <?= $error; ?>
        </div>
        <?php endif; ?>

        <?php if (!empty($exito)): ?>
            <div class="resultado alerta-exito">
                <?= $exito; ?>
            </div>
        <?php endif; ?>
    </body>
</html>
```

Herramienta Recortes

Por otro lado, después del formulario tenemos un bloque if de php, donde dependiendo de si se ha recibido el mensaje de error o de éxito se muestra uno u otro.

Si entramos al funcionamiento de cifrar, lo primero escrito es el bloque de código PHP que recoge los datos, y a su vez si no hay datos no permite entrar en cifrar.php. El usuario primero debe pasar por index.php e introducir datos.

```php
<?php
$mensaje = "";
$semilla = "";

if (isset($_POST['mensaje'], $_POST['semilla'])) {
    $mensaje = trim($_POST['mensaje']);
    $semilla = $_POST['semilla'];
} else {
    header("Location: index.php?error");
    exit();
}
```

Posteriormente, está el código de todo el funcionamiento donde se producen las validaciones y las muestras de mensajes. El alfabeto ha sido creado mediante un array. Para cifrar el mensaje, primero se busca la posición de cada letra dentro del alfabeto mediante array_search(). Después, se suma a esa posición el valor de la semilla para obtener la nueva posición. Si se supera el final del alfabeto, se vuelve al principio para continuar el desplazamiento. Por último, se sustituye cada letra por la que ocupa la nueva posición y se forma el mensaje cifrado. Para el descifrado es igual pero se resta a esa posición el valor de la semilla.

```php
$error = "";
$exito = "";

if (preg_match('/^[a-zA-ZñÑ]+$/u', $mensaje)) {
    if ($semilla < 1) {
        $error = "La semilla no puede ser un numero negativo.";
    } else {
        $alfabeto = [
            "A", "B", "C", "D", "E", "F", "G",
            "H", "I", "J", "K", "L", "M", "N",
            "Ñ", "O", "P", "Q", "R", "S", "T",
            "U", "V", "W", "X", "Y", "Z"
        ];

        $mensaje = mb_strtoupper($mensaje, "UTF-8");
        $mensajeCifrado = "";

        for ($i = 0; $i < mb_strlen($mensaje, "UTF-8"); $i++) {
            $letra = mb_substr($mensaje, $i, 1, "UTF-8");
            $posicion = array_search($letra, $alfabeto);
            $nuevaPosicion = $posicion + $semilla;

            if ($nuevaPosicion > 26) {
                $nuevaPosicion = 0 + ($nuevaPosicion - 27);
            }

            $letraCifrada = $alfabeto[$nuevaPosicion];
            $mensajeCifrado .= $letraCifrada;
        }

        $exito = "Mensaje recibido: " .
        $mensaje . "<br>Semilla recibida: " .
        $semilla . "<br>Mensaje cifrado: " . $mensajeCifrado;
    }
} else {
    $error = "El mensaje no puede tener caracteres que no sean letras.";
}
```

Para terminar se pasa el mensaje de éxito, si todas las validaciones han sido correctas y el mensaje ha sido cifrado, o el de error en caso contrario mediante la url a la página de index de nuevo. Para entonces mostrar visualmente para el usuario el mensaje correspondiente.

```php
if ($error != "") {
    header("Location: index.php?error=" . urlencode($error));
} else {
    header("Location: index.php?exito=" . urlencode($exito) . "&mensaje=" . urlencode($mens
}
exit();
```

El archivo descifrar.php funciona exactamente igual pero cambiando la lógica. En vez de avanzar las posiciones de la semilla hacia delante, las retrocede restando la semilla.

```php
$nuevaPosicion = $posicion - $semilla;

if ($nuevaPosicion < 0) {
    $nuevaPosicion = 27 - ($nuevaPosicion * -1);
}
```

2. CIFRAR Y DESCIFRAR VERSIÓN 2

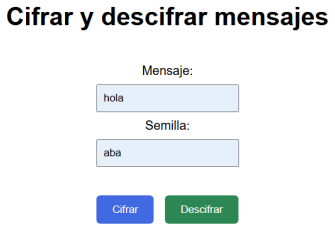
Esta aplicación es un cifrado de mensajes. El usuario introduce el mensaje que desee y la semilla, ambos textos deben ser todo letras. No están permitidos los espacios, debe ser un mensaje únicamente de letras y sin espacios.

2. 1. MANUAL DE USUARIO

La aplicación realizada le pide al usuario el mensaje a introducir y la semilla que desea para proceder a cifrar el mensaje. A su vez también le da la opción de elegir si quiere cifrar o descifrar el mensaje.



Una vez haya introducido el mensaje y la semilla (necesario ambos campos) tendrá que pulsar el botón de cifrar y aparecerá el resultado del mensaje cifrado debajo de los botones. Y tras cifrar el mensaje si vuelve a darle al botón de descifrar le aparecerá el mensaje descifrado de nuevo.







Si en el mensaje se introduce cualquier otro carácter que no sean letras saltará un mensaje de no válido.





Se mostrarán los errores en color rojo para que se distingan con mayor facilidad.

El usuario tendrá que introducir valores nuevos válidos en los campos y pulsar nuevamente el botón para cifrar o descifrar el mensaje que desee.

2. 2. MANUAL TÉCNICO

El funcionamiento de este ejercicio es exactamente igual que la versión 1. Solo cambia la lógica dentro de los archivos cifrar.php y descifrar.php para que la semilla sean letras y a partir de ahí cifre el mensaje.Este es el bloque lógico de cifrar.php.

```php
if (preg_match('/^[a-zA-ZñÑ]+$/u', $mensaje)) {
    if(preg_match('/^[a-zA-ZñÑ]+$/u', $semilla)){

                $alfabeto = [
                    "A", "B", "C", "D", "E", "F", "G",
                    "H", "I", "J", "K", "L", "M", "N",
                    "Ñ", "O", "P", "Q", "R", "S", "T",
                    "U", "V", "W", "X", "Y", "Z"
                ];

                $mensaje = mb_strtoupper($mensaje, "UTF-8");
                $mensajeCifrado = "";

                for ($i = 0; $i < mb_strlen($mensaje, "UTF-8"); $i++) {
                        $letraMen= mb_substr($mensaje, $i, 1, "UTF-8");
                        $posSemilla = $i % mb_strlen($semilla, "UTF-8");
                        $letraSem = mb_strtoupper(mb_substr($semilla, $posSemilla, 1, "UTF-8"), "UTF-8");
                        $posicion = array_search($letraMen, $alfabeto);
                        $posicionSem = array_search($letraSem, $alfabeto);
                        $nuevaPosicion = ($posicion + 1) + ($posicionSem + 1);
                        if ($nuevaPosicion > 27) {
                            $nuevaPosicion = $nuevaPosicion % 27;
                        }
                        $letraCifrada = $alfabeto[$nuevaPosicion - 1];
                        $mensajeCifrado .= $letraCifrada;
                }

                $exito= "Mensaje recibido: " . $mensaje . "<br>Semilla recibida: " . $semilla . "<br>Mensaje cifrado: " . $mensajeCifrado;

    }else {
        $error="La semilla no puede tener caracteres que no sean letras.";
    }
} else {
    $error="El mensaje no puede tener caracteres que no sean letras.";
}
```

Este es el bloque lógico de descifrar.php.

```php
if (preg_match('/^[a-zA-ZñÑ]+$/u', $mensaje)) {
if(preg_match('/^[a-zA-ZñÑ]+$/u', $semilla)){
        $alfabeto = [
            "A", "B", "C", "D", "E", "F", "G",
            "H", "I", "J", "K", "L", "M", "N",
            "Ñ", "O", "P", "Q", "R", "S", "T",
            "U", "V", "W", "X", "Y", "Z"
        ];

        $mensaje = mb_strtoupper($mensaje, "UTF-8");
        $mensajeDescifrado = "";

        for ($i = 0; $i < mb_strlen($mensaje, "UTF-8"); $i++) {
            $letra= mb_substr($mensaje, $i, 1, "UTF-8");
            $posSemilla = $i % mb_strlen($semilla, "UTF-8");
            $letraSem = mb_strtoupper(mb_substr($semilla, $posSemilla, 1, "UTF-8"), "UTF-8");
            $posicionSem = array_search($letraSem, $alfabeto);
            $posicion = array_search($letra, $alfabeto);
            $nuevaPosicion = ($posicion + 1) - ($posicionSem + 1);
            if ($nuevaPosicion < 1) {
                $nuevaPosicion = $nuevaPosicion + 27;
            }
            $letraDescifrada = $alfabeto[$nuevaPosicion - 1];
            $mensajeDescifrado .= $letraDescifrada;
        }

        $exito= "Mensaje recibido: " . $mensaje . "<br>Semilla recibida: " . $semilla . "<br>Mensaje descifrado: " . $mensajeDescifrado;
    }else {
        $error="La semilla no puede tener caracteres que no sean letras.";
    }
} else {
    $error="El mensaje no puede tener caracteres que no sean letras.";
}
```

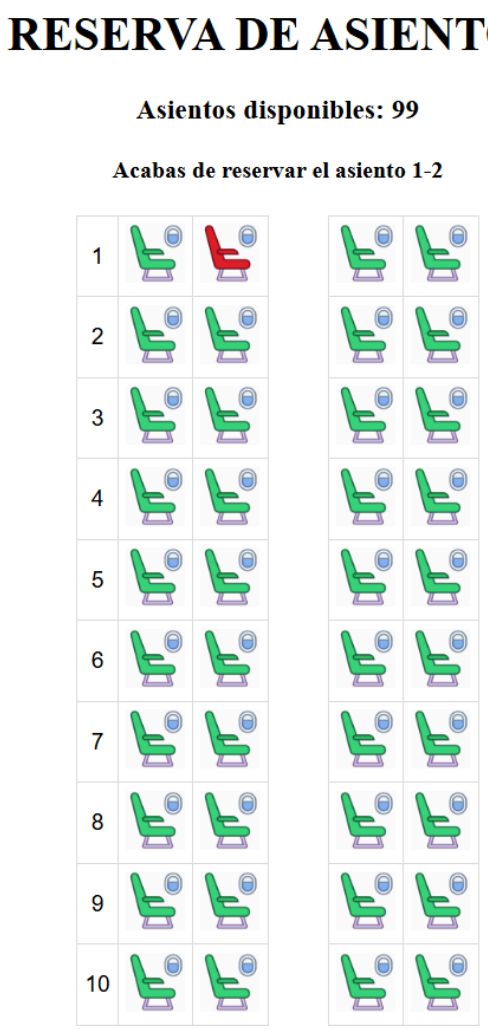
Para cifrar el mensaje, se busca la posición de cada letra de la semilla en el alfabeto. Esa posición se utiliza como desplazamiento (se suma a la posición de la letra del mensaje) para cifrar la letra correspondiente del mensaje. Se repite el proceso con cada pareja de letras hasta obtener el mensaje cifrado. Para descifrar el mensaje, se resta esa posición a la de la letra cifrada para recuperar la letra original. Se repite el proceso con cada pareja de letras hasta obtener el mensaje descifrado.

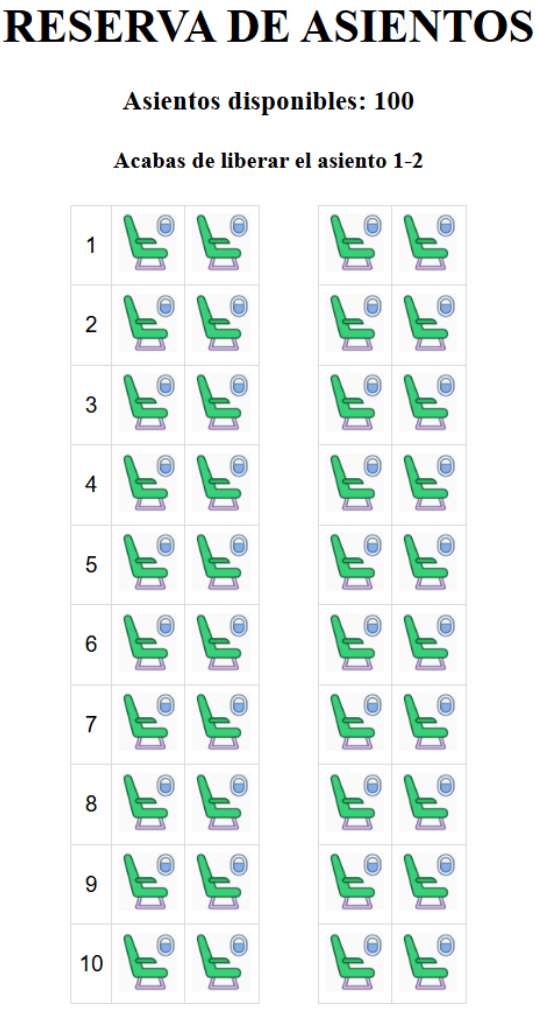
3. RESERVA DE ASIENTOS DE AVIÓN

Esta aplicación es un sistema de reservas de asientos de avión. El usuario hace click en el asiento que desea reservar y este se cambia a color rojo simulando que está ocupado.

3. 1. MANUAL DE USUARIO

La aplicación muestra visualmente como si fuese el plano de un avión para permitir reservar asientos. Se debe clicar en el asiento que desee para que este pase a estar reservado y cambie a color rojo. Si éste ya estaba en color rojo y es pulsado, pasa a estar libre cambiando a color verde de nuevo.

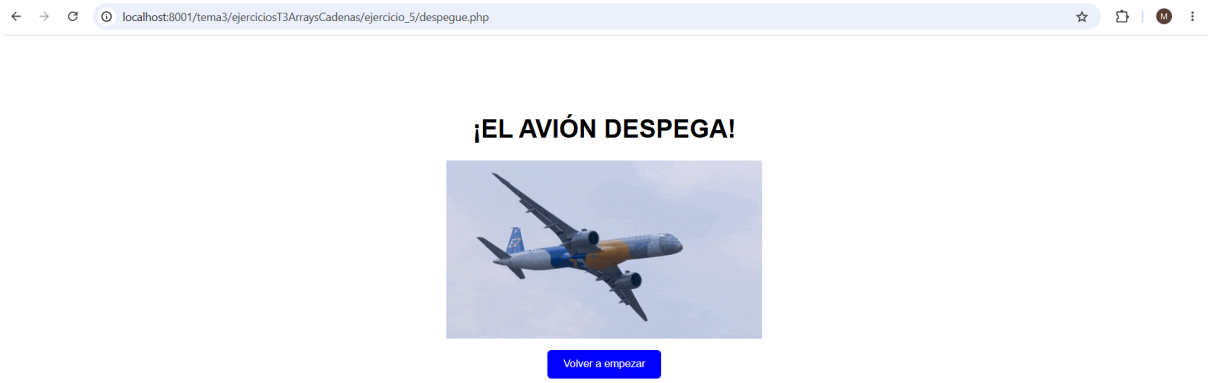




En la parte superior se muestran los asientos disponibles. Al igual que cada vez que pulsas un asiento se muestra un mensaje del número de asiento que has liberado o ocupado, dependiendo en qué estado estaba.



Por último, si los 100 asientos acaban siendo ocupados, se muestra un mensaje de despegue del avión, con un botón de opción para reiniciar la aplicación.



3.2 . MANUAL TÉCNICO

El ejercicio está separado en 2 archivos: asientos.php que es donde está toda la lógica y parte visual, y despegue.php que simplemente es la pantalla que aparece de despegue cuando se reservan los 100 asientos. Al principio del archivo asientos.php iniciamos una session que nos servirá para hacer un array de asientos ocupados. Con un isset validamos la entrada de datos de fila y columna, que equivalen a la fila y la columna que el usuario ha pulsado y los guardamos en variables formando así el id del asiento.

```php
<?php
session_start();
if (isset($_POST['fila']) && isset($_POST['columna'])) {
    $mensaje="";
    $fila_pulsada = $_POST['fila'];
    $columna_pulsada = $_POST['columna'];
    $id_asiento = $fila_pulsada . "-" . $columna_pulsada;

    $asientosTotales=100;

    if (isset($_SESSION['ocupados'][$id_asiento])) {
        unset($_SESSION['ocupados'][$id_asiento]);
        $mensajeLibre="Acabas de liberar el asiento " . $id_asiento;
    } else {
        $_SESSION['ocupados'][$id_asiento] = true;
        $mensaje="Acabas de reservar el asiento " . $id_asiento;
    }
}
```

Posteriormente, realizamos un isset para comprobar si en la sesión ese asiento pulsado está en el array de ocupados que creamos con la session. Si está dentro del array de ocupados, el programa lo saca del array y lo libera. Si no está dentro del array de ocupados, el programa lo mete en ese array y lo sitúa como

reservado/ocupado. Después de ese código tenemos la función de obtenerImagen, que dependiendo de si el asiento pulsado (id_asiento) está dentro del array de ocupados o no, muestra la imagen del asiento rojo o verde.

```php
function obtenerImagen($id_asiento){
    if (isset($_SESSION['ocupados'][$id_asiento])) {
        return "asientoRojo.jpg";
    } else {
        return "asientoVerde.jpg";
    }
}
```

Por último dentro de la lógica del código php, tenemos el contador de asientos disponibles, que calcula los asientos ocupados mediante count del array de ocupados. Si el valor de asientos disponibles llega a 0, el programa manda al archivo despegue.php y este se muestra en pantalla.

```php
$asientos_totales = 100;
$asientos_ocupados = 0;

if (isset($_SESSION['ocupados'])) {
    $asientos_ocupados = count($_SESSION['ocupados']);
}

$asientos_disponibles = $asientos_totales - $asientos_ocupados;
if ($asientos_disponibles==0){
    header("Location: despegue.php");
    exit;
}
?>
```

Por otro lado, en la parte de código html está el if que muestra el mensaje de asiento reservado o libre dependiendo de lo que ha sucedido. Y los valores de fila y columna de cada asientos son enviados mediante input hidden. Cada asiento tiene su propio form.

```php
<h1>RESERVA DE ASIENTOS</h1>
<h3>Asientos disponibles: <?= $asientos_disponibles ?></h3>
<h4><?php if (!empty($mensaje)): ?>
        <div class="asientoReservado">
            <?= $mensaje; ?>
        </div>
    <?php endif; ?>
    <?php if (!empty($mensajeLibre)): ?>
        <div class="asientoLibre">
            <?= $mensajeLibre; ?>
        </div>
    <?php endif; ?>
    </h4>
    <table>
        <tbody>
            <tr>
                <td>1</td>
                <td>
                    <form action="<?= $_SERVER['PHP_SELF'] ?>" method="post">
                    <input type="image" src="<?= obtenerImagen('1-1') ?>" alt="Asiento">
                    <input type="hidden" name="fila" value="1">
                    <input type="hidden" name="columna" value="1">
                    </form>
```